布尔函数与向量值函数
科研技能 · 第一版

DeepSeek Harness 安装与使用教程

从安装仓颉到科研复现

公开版本：V1 / 1.0.0
冻结开发基线：0.2.2
2026 年 10 月 27 日

完成本教程后，你应能：安装仓颉和科研 Skill，构造一个有已知答案的函数，计算全分量指标，保存完整实验与日志，核对哈希并重算结果。教程给出可复制命令、预期结果和常见故障处理。

本项目不是原书 PDF 副本。第一版保留 16 张能力卡和 8 个标准库运行程序；内部数学程序版本仍为 0.2.0，工作流为 0.2.2，公开发行名称为 1.0.0，三者含义不同。

目录

第一章 第一步：为 DeepSeek 安装仓颉 cangjie-skill **3**

1.1 先分清模型、网页和 Harness 3

1.2 安装并验证 Harness 宿主 3

1.3 下载仓颉官方 DeepSeek 插件并核验 4

1.4 安装仓颉并启动网页宿主 4

1.5 仓颉作者端依赖 5

第二章 第二步：安装第一版 DeepSeek 插件 **6**

2.1 选择对应发行物 6

2.2 第一条科研请求 6

2.3 独立于模型的本地验算路径 6

2.4 适配验证的范围 7

第三章 从一个 JSON 开始理解输入合同 **8**

3.1 整数表怎样表示函数 8

3.2 域模型、谱和计数不能混淆 8

第四章 完整实验：GF(8) 立方函数 **9**

4.1 保存项目文件 9

4.2 核对预期结果 9

4.3 只算核心指标或自行给表 10

第五章 日志、哈希与重算：怎样验收结果 **11**

5.1 先看完成清单，再核对哈希 11

5.2 阶段日志究竟检查什么 12

5.3 失败命令也保存日志 12

第六章 进一步使用：迹构造、等价证书与编码桥梁 **13**

6.1 绝对迹和多项式构造 13

6.2 给定 CCZ 图变换证书 13

6.3 函数生成码的科研用法 14

第七章 第一版 16 张能力卡：何时调用 **15**

7.1 一条有效研究请求应含哪些条件 15

第八章 常见错误、升级与维护 **16**

8.1 作者端：从仓颉源重新编译 16

8.2 TeX 怎样编译成 PDF 17

8.3 后续 GitHub 上传继续沿用的流程 17

第九章 验收清单与来源 **18**

第一章　第一步：为 DeepSeek 安装仓颉 cangjie-skill

1.1　先分清模型、网页和 Harness

本安装版面向 **DeepSeek Harness（dsh）**。它是可加载技能并调用本地工具的宿主；普通 DeepSeek 网页聊天不等于本地程序执行环境。本章先准备 dsh，再安装仓颉插件。官方仓颉地址为https://github.com/kangarooking/cangjie-skill，注意实际拼写是 cangjie。

准备 Python3.10+ 与 Node.js24 系列或满足宿主要求的更高版本。Windows 打开 PowerShell 检查：

```
py -3 --version
node --version
npm --version
```

分别应出现版本号。若缺少 Python 或 Node，先从各自官方网站安装并重新打开终端；不要把本项目的 TGZ 当作 dsh 宿主安装器。Node 官网下载：https://nodejs.org/。

1.2　安装并验证 Harness 宿主

本教程按 2026-10-27 核对到的官方 npm 版本 `0.2.1-rc.2` 给出可重复安装命令：

```
npm install -g @deepseek-ai/dsh@0.2.1-rc.2
dsh --help
```

若不做全局安装，可使用 `npx @deepseek-ai/dsh@0.2.1-rc.2` 代替下文的 dsh 命令前缀。Harness 处于开发预览，未来可能变更命令或插件合同；以官方仓库https://github.com/deepseek-ai/deepseek-harness及安装后的 help 为准，升级后重新验收，不直接套用旧界面截图。

1.3 下载仓颉官方 DeepSeek 插件并核验

创建下载目录，打开仓颉 v2.5.0 的 Release 页，下载以下两个文件：

```
dsh-cangjie-skill-2.5.0.tgz
dsh-cangjie-skill-2.5.0.tgz.sha256
```

它们对应的官方 URL 为：

```
https://github.com/kangarooking/cangjie-skill/releases/download/v2.5.0/dsh-cangjie」
↪  -skill-2.5.0.tgz
https://github.com/kangarooking/cangjie-skill/releases/download/v2.5.0/dsh-cangjie」
↪  -skill-2.5.0.tgz.sha256
```

在文件所在目录运行：

```
Get-FileHash .\dsh-cangjie-skill-2.5.0.tgz -Algorithm SHA256
```

与下载的校验文件比较。本次实际核验的 SHA256 为：

```
b565c693eaf7bc24f09bde5df15e1c7e63fee504773f98288e909e7b3086ddbd
```

上游如果重新发布不同文件，应先核对上游发布说明和新校验值，不强行认为旧值仍适用。

1.4 安装仓颉并启动网页宿主

在 TGZ 所在目录执行：

```
dsh plugin --profile web add .\dsh-cangjie-skill-2.5.0.tgz
dsh web
```

命令以仓颉官方 DeepSeek 安装说明为依据。web 是本教程使用的配置档；若使用已安装的桌面宿主，必须选择该宿主实际使用的 profile，不能把“已写入 web”当成“desktop 也已加载”。

在宿主首次启动提示或模型配置处选择可用的模型服务商，使用你自己的账户授权或 API 凭据；支持哪些方式依宿主版本而定。本包不含密钥或额度。密钥只进入宿主配置，不写进本项目、TeX、GitHub 或日志。无需为验证安装而运行长时间付费研究任务。

启动新任务后输入：“使用 cangjie-skill，先定位其 SKILL.md 及资源目录，检查我给出的 PDF 路径、作者和研究目标，然后提出蒸馏计划。”能加载技能并定位资源是验收点；本项目不分发原书，使用已蒸馏成果不要求再次蒸馏原书。

1.5 仓颉作者端依赖

仓颉脚本需要 Python3.10+ 和 PyYAML。先让宿主给出仓颉的实际资源基目录，再在该目录创建隔离环境并执行 doctor，避免猜测插件管理器的内部缓存路径：

```
py -3 -m venv .venv
& .\.venv\Scripts\python.exe -m pip install pyyaml
& .\.venv\Scripts\python.exe .\scripts\cangjie.py doctor
```

以上是在仓颉资源根目录运行。科研数学程序本身只使用 Python 标准库。

第二章　第二步：安装第一版 DeepSeek 插件

2.1　选择对应发行物

从https://github.com/WEveboy/The-Theory-of-Error-Correcting-Codes-skill/releases/tag/edition-v1.0.0下载 boolean-vector-research-DeepSeek-1.0.0.zip 及 SHA256SUMS.txt。ZIP 内含原生 dsh-boolean-vector-research-1.0.0.tgz、两份教程和说明；若只安装插件，可直接下载同名 TGZ。

在下载目录执行 Get-FileHash 比较同名资产的 SHA256。ZIP 的校验值对应 ZIP，TGZ 的校验值对应 TGZ，不能混用。解压 ZIP 后在 TGZ 所在目录运行：

```
dsh plugin --profile web add .\dsh-boolean-vector-research-1.0.0.tgz
dsh web
```

插件注册名为 dsh-boolean-vector-research，模型可调用的技能名为 boolean-vector-research。它与仓颉是两个独立插件。添加成功后用新任务验收，并确认当前启动的 profile 就是刚才安装的 profile。

2.2　第一条科研请求

> 使用 boolean-vector-research 技能。先定位实际资源目录，再构造 4 变量二次 bent，计算全部 Walsh 谱、非线性度、极化秩与函数生成码。把结果保存在工作目录，给出完整输入和可复算命令。

预期 $f = x_0x_1 + x_2x_3$，NL=6、最大谱幅 4、极化秩 4、生成码秩 6、最小非零重量 6。DeepSeek 是否实际调用工具，要查看工具执行记录与结果文件，不能只凭对话中的一段数字验收。

2.3　独立于模型的本地验算路径

如果暂时没有配置模型，也能核对数学工具。将 TGZ 解压到新目录：

```
New-Item -ItemType Directory -Force C:\SkillLab\deepseek-runtime | Out-Null
tar -xzf .\dsh-boolean-vector-research-1.0.0.tgz -C C:\SkillLab\deepseek-runtime
$skill = "C:\SkillLab\deepseek-runtime\package\skills\boolean-vector-research"
$res = Join-Path $skill "resources"
New-Item -ItemType Directory -Force C:\SkillLab\work | Out-Null
Set-Location C:\SkillLab\work
py -3 "$res\research_core.py" demo --case scalar-bent --out bent.json
```

系统没有 tar 时使用能够解压 TGZ 的工具，先解外层 gzip 再解 tar，最终应看到 package/skills/boolean-vector-research。该路径用于本地独立验算，不等于已经向 Harness 注册插件。宿主实际加载时以其提供的 resourceBase 为准。

2.4 适配验证的范围

第一版的 DeepSeek 适配器检查了注册合同、资源定位、生命周期返回值、包配置与两平台资源 SHA256 相同。数学工具另做完整测试。没有用本机密钥发起付费模型会话，因此插件契约测试不冒充真实在线模型的全链路评测。

第三章　从一个 JSON 开始理解输入合同

3.1　整数表怎样表示函数

对 $F:\mathbb{F}_2^n\to\mathbb{F}_2^m$，table[x] 表示 $F(x)$ 的整数编码；输入整数的第 i 位是 x_i，输出第 i 位是 F_i。当前支持 $1\le m\le n$。例如：

```
{"n":2,"m":1,"table":[0,0,0,1]}
```

这是 x_0x_1。长度必须等于 2^n，输出必须在 0 到 2^m-1 之间。JSON 的 true/false 和浮点不能冒充整数；JSON 用英文双引号，不带尾逗号。文件名要确实以.json 结尾，避免隐藏的.txt 后缀。

3.2　域模型、谱和计数不能混淆

有限域整数是多项式基下的编码。模多项式 11 表示 x^3+x+1；域表示改变后，列表不一定保持相同。Walsh 使用未归一化整数和；DDT 按所有有序输入 x 计数；DLCT 使用计数偏差 $\#\text{equal}-2^{n-1}=AC/2$，再除 2^n 才是概率偏差。

向量性质必须检查所有非零输出掩码 b 的分量 $b\cdot F$，不能只检查坐标。经典 BCT 只对方形置换计算；非置换返回 not_applicable。相同谱/DDT 不变量不能证明 CCZ 等价。

第四章　完整实验：GF(8) 立方函数

4.1　保存项目文件

先完成本平台章节中的 `$res` 设置，在独立工作目录保存 project.json：

```
{
  "construction": {
    "kind": "gold",
    "params": {"n": 3, "k": 1, "polynomial": 11}
  },
  "advanced": true,
  "sbox": true
}
```

可在 PowerShell 用单引号 here-string 保存，或用文本编辑器另存为 UTF-8。然后运行：

```
py -3 "$res\research_workflow.py" --input .\project.json --out-dir .\experiment-001
```

输出目录必须不存在。已有 experiment-001 时改用 experiment-002，保留旧实验作比较。

4.2　核对预期结果

本例 $F(x) = x^3$ 在 GF(8) 上是置换，NL=2、差分均匀性 $\delta = 2$、boomerang 均匀性为 2。它不是 bent。小维数值是已知基准，不能据此宣布新族或当前最优。

目录应包含 project.json、function.json、analysis.json、sbox.json、summary.md、environment.json、execution.jsonl 和 manifest.json。function 保存完整表；analysis 保存全分量结果；sbox 保存 BCT/DLCT 和传播检查；其余记录输入、环境和执行证据。

4.3 只算核心指标或自行给表

```
py -3 "$res\research_core.py" analyze --input function.json --out core.json
py -3 "$res\sbox_criteria.py" analyze --input function.json --out sbox-full.json
↪  --include-tables
```

–advanced 才请求精确 AI 和二阶非线性度：AI 仅 $n \le 6$，精确二阶 NL 仅 $n \le 4$；超维字段为 not_computed，不能把它写成 0。新扩展通常 $n \le 8$，核心默认 $n \le 8$，允许显式放宽但仍受谱条目与工作预算限制。

第五章　日志、哈希与重算：怎样验收结果

5.1　先看完成清单，再核对哈希

manifest 最后写入；只看目录存在不够。先检查 status 为 complete，再核对 files 里的 SHA256。推荐直接运行只读审计器：

```
py -3 "$res\experiment_audit.py" --directory .\experiment-001 --out
↪  .\audit-001.json
py -3 "$res\experiment_audit.py" --directory .\experiment-001 --recompute --out
↪  .\replay-001.json
```

报告必须是实验目录外的新文件；程序不会覆盖旧报告。支持 0.2.0、0.2.1、0.2.2 历史实验。第一版沿用 0.2.2 工作流版本，EDITION.json 另记公开版本 1.0.0。

状态	应如何解释
hash_consistent	文件与当前无签名清单一致；不是记录者身份或数值真实性的认证。
recomputed_match	按原参数使用当前程序重新生成数学结果后匹配。重算重用同一程序，不是独立算法证明。
hash_mismatch	文件字节与清单哈希不同，停止重算并检查来源。
content_mismatch	输入关联、日志协议或重算结果不一致；查看 failures 及 witness.pointer。
退出码 2	缺文件、损坏 JSON、重复键、非有限数、非法时间/路径、环境合同或预算错误。

退出码 0 表示通过，1 表示检测到哈希或内容差异。PowerShell 运行后用 `$LASTEXITCODE` 查看退出码，不只看终端有没有红字。

5.2 阶段日志究竟检查什么

execution.jsonl 每行一条事件，包含整数 sequence、显式 UTC 时间、elapsed_seconds、stage 和 status。审计器检查阶段是否与启用/跳过 S-box 的项目配置一致、序号是否连续、耗时是否为有限非负且不倒退的数。墙上时钟可能校正，不能把它代替单调耗时。

environment.json 记录 Python、平台和运行程序 SHA256。runtime_fingerprints_match 独立报告代码指纹是否相同；版本改变不自动等于数学结果错误。重算发现不同会给首个 JSON 指针，例如/nonlinearity，或/components/0/walsh/3，并给差异类型和限长预览。

5.3 失败命令也保存日志

数学计算失败时工作流不创建成功实验目录；需要额外保存终端输出：

```
py -3 "$res\research_workflow.py" --input project.json --out-dir experiment-002
↪  2>&1 | Tee-Object -FilePath command-002.log
$LASTEXITCODE
```

先选择不存在的日志名，避免手工命令覆盖旧记录。成功实验内部阶段日志和失败命令输出是不同证据。不要把未计算、未调用或未完成的项写入“已通过”。

第六章　进一步使用：迹构造、等价证书与编码桥梁

6.1　绝对迹和多项式构造

保存 field.json：

```
{"n":3,"polynomial":11,"terms":[{"coefficient":1,"exponent":3}]}
```

```
py -3 "$res\field_functions.py" trace --input field.json --out trace.json
py -3 "$res\research_core.py" analyze --input trace.json --out trace-analysis.json
```

将 trace 改成 polynomial 得到向量值多项式表；dual-basis 使用 n、polynomial 和可选 basis，给迹配对矩阵、对偶基和坐标恢复。指数 0 是常数项，采用 $0^0 = 1$ 的多项式约定，不对指数盲目模 $2^n - 1$。构造表始终按标准多项式基索引。

6.2　给定 CCZ 图变换证书

保存 certificate.json，交换输入输出的简单基准：

```
{"n":2,"m":2,"table":[0,1,2,3],"graph_rows":[4,8,1,2],"graph_shift":0,"target":[0,」
↪  1,2,3]}
```

```
py -3 "$res\graph_equivalence.py" certificate --input certificate.json --out
↪  certificate-result.json
```

图编码为 $z = x \,|\, (F(x) \ll n)$，矩阵行是位掩码，低 n 位为新输入。矩阵必须可逆；投影碰撞会返回 not_function_graph 与见证。给了 target 还要查看 matches_target。该程序不搜索未知 CCZ 变换，也不做完整分类。

prefilter 输入 n、m、left 和 right 两张表。完整图谱/DDT 直方图不同可以否定 CCZ；相同仅 inconclusive。次数、BCT 和 DLCT 不作为本工具承诺的 CCZ 不变量。

6.3 函数生成码的科研用法

工具用常数、输入坐标和输出坐标生成 C_F。只有实际秩为 $n+m+1$，才满足本工具用最小非零重量联系 NL 的满秩前提；不能把提供的行数当秩。码枚举还有 rank 和长度预算。先核对 generated_code.full_rank_bridge_applicable，再解释距离和重量分布，最后按明确条件连接对偶枚举器、设计或 OA。

第七章　第一版 16 张能力卡：何时调用

能力	合适任务与输出
表示与规格	位序、真值表、ANF、域模型、维数与预算检查。
标量谱/RM 陪集	Walsh、NL、最近仿射函数、RM1 距离证据。
二次型	极化秩、根空间、导数核与二次谱条件。
bent 构造	二次与 MM 构造、前提证明和全谱核验。
向量指标	全非零分量、DDT、最弱分量和最坏方向。
函数生成码	秩门、有限重量枚举与 NL 桥梁。
高阶不变量	自相关、CI/韧性、线性结构，小维 AI 和 NL2。
迹与幂搜索	固定域模型的 Gold/幂候选、显式搜索集合。
EA 证书	给定 EA 矩阵/平移逐点验证与目标匹配。
枚举器与界入口	精确 MacWilliams 变换、非负整系数必要条件；非通用 LP。
设计与 OA	给定块/行的精确计数及失败见证。
序列相关	周期与 xor 群作用分别定义的自/互相关。
有限域函数	多项式、绝对迹、对偶基和坐标恢复。
S-box 准则	经典置换 BCT、DLCT、坐标 SAC 和全分量 PC。
CCZ 图证书	给定图仿射证书与必要不变量预筛。
完整实验	构造到指标、阶段日志、环境指纹、哈希与重算。

7.1　一条有效研究请求应含哪些条件

建议按“问题与目标指标、n/m 和表示、构造参数、预算、需要的证明、输出路径”组织请求。例如：在 GF(8) 明确模多项式 11，搜索指数 1、3、5，要求置换且差分均匀性 2，保存全部候选和失败原因；不把有限候选通过写成新族。

有研究潜力的组合：二次型空间到向量 bent 再到少重量码和支持集检查；Gold/迹构造到全谱/DDT/BCT 后再做证书核验；谱约束到韧性/AI/高阶距离。每条桥梁单独核对前提，保留尚待证明的义务。

第八章　常见错误、升级与维护

1. 找不到 python：使用实际可用的 py -3 或解释器完整路径，重新打开终端。不要把安装目录路径直接当成命令。
2. 找不到 SKILL：检查有没有多套一层目录，资源是否完整，宿主是否重新加载；DeepSeek 还要确认 profile 一致。
3. JSON 报错：检查英文双引号、尾逗号、长度、值域和重复键；true 不能作为整数。
4. 输出目录已存在：换新实验目录，保留旧结果。报告文件必须在实验目录外且尚不存在。
5. 预算不足：先减小问题规模；确需增大时显式调整–max-work，并记录参数，不能用采样冒充精确结论。
6. 指纹不同但重算匹配：如实报告运行环境变化与数学匹配，保留原环境文件，不改写历史归档。
7. 已装旧版：Codex 使用显式备份升级；DSH 按宿主插件管理方式升级后新任务验收。不要在两个发现目录放同名入口。

8.1　作者端：从仓颉源重新编译

仓库 source 保存能力卡与 verified.yaml，skills 保存冻结运行资源。需要修改技能时，编辑源卡和对应程序/测试，再在仓库根目录、装有 PyYAML 的 Python 环境中运行：

```
python scripts/rebuild_skill.py --cangjie C:\path\to\cangjie-skill
```

生成结果在 build/recompiled-skill，不自动覆盖用户安装。该编译动作不等于通过来源验证、数学测试、宿主发现或文献查新，修改后仍应完成相应验收。

8.2 TeX 怎样编译成 PDF

两份导出的.tex 是完整独立源文件。使用 XeLaTeX 和 CTeX/Fandol 字体；在教程目录分别执行两次以更新目录与链接：

```
xelatex -interaction=nonstopmode -halt-on-error Codex-First-Edition.tex
xelatex -interaction=nonstopmode -halt-on-error Codex-First-Edition.tex
xelatex -interaction=nonstopmode -halt-on-error DeepSeek-First-Edition.tex
xelatex -interaction=nonstopmode -halt-on-error DeepSeek-First-Edition.tex
```

若没有 TeX 环境，可安装 TeX Live，或将独立 TeX 源上传支持 XeLaTeX 的编辑器并选择 XeLaTeX 编译器。修改后要看编译日志并渲染检查分页、命令和中文，不只检查 PDF 文件存在。

8.3 后续 GitHub 上传继续沿用的流程

先读 AGENTS.md 和 docs/UPLOAD_WORKFLOW.md，核对远端 main；只提交本次源码、测试、范例、TeX 与最终手册 PDF，保留他人改动。排除 build/dist、缓存、私人材料和凭据。完成数学/安装器/适配器测试、文档链接检查与 PDF 编译检查；上传后核对提交 SHA 和 CI。版本化安装包、源码 ZIP、校验文件作为 Release 资产发布并验收下载。本项目未新增 LICENSE。

第九章　验收清单与来源

1. 仓颉已加载，完整 SKILL 与资源可定位；doctor 实际运行，或明确未运行原因。
2. 对应平台的第一版安装正确，版本信息与 EDITION 清单相符。
3. 二次 bent 基准与 GF(8) 立方基准得到预期数值。
4. 实验输入、完整表、全分量指标、环境、日志及清单保存在工作目录。
5. 审计状态、退出码、未计算项和失败见证均已查看；重算与哈希状态分别解释。
6. 论文结论另给全参数条件证明；新颖性另做原始文献查新，不以本教程有限算例代替。

项目：https://github.com/WEveboy/The-Theory-of-Error-Correcting-Codes-skill。仓颉：https://github.com/kangarooking/cangjie-skill。Codex 技能文档：https://learn.chatgpt.com/docs/build-skills。DeepSeek Harness：https://github.com/deepseek-ai/deepseek-harness。DSH 本次锁定版本的官方 npm 元数据：https://registry.npmjs.org/@deepseek-ai/dsh/0.2.1-rc.2。

研究中的 B（原书）、D（独立推导）、E（现代来源）、H（研究假设）分别标注。原书重点章节和来源核读范围见 Skill 内 proof_ledger 与 overview；不宣称逐页逐题完成全书验证。